Coding and Software Cookbook

Data Analysis Companion v3

Python and R workflow companion for the Data Analysis textbook

Contents

Coding and Software Cookbook

This companion collects implementation patterns that support the Data Analysis textbook. The main chapters develop concepts, assumptions, interpretation, and mathematical structure. This cookbook keeps the package-specific Python and R workflows close at hand so students can turn those concepts into reproducible analyses during labs and assignments.

The cookbook is organized by analytic workflow rather than by lecture order. Each block names the chapter material it supports, but the recipes are meant to be reused across projects. Code is not a substitute for judgment: it can compute summaries, fit models, and extract diagnostics, but the analyst still decides whether the result is meaningful and defensible.

1 How to Use This Cookbook

Use this companion after reading the corresponding chapter or while working through a lab.

- Start with the question: What are you trying to learn, predict, or explain?
- Identify the response, predictors, units, and variable types before running a model.
- Use each code block as a pattern. Rename variables, check units, and verify assumptions before reusing it.
- Inspect output before reporting it. A model summary is not an analysis until you interpret it.
- Preserve enough code and notes that another analyst can reproduce the result.

Workflow block	Main chapter support	Main purpose
Project setup	Chapters 1–3	Organize a reproducible project before analysis begins
Data quality and cleaning	Chapter 3	Inspect columns, missingness, placeholder values, data types, impossible values, and cleaned-data outputs
EDA patterns	Chapter 4	Compute summaries, create plots, and inspect correlations before modeling
SLR fitting	Chapter 5	Fit simple regression, compute coefficients manually, and extract residuals
SLR fit and inference	Chapter 6	Compute R^2, F-statistics, p-values, RSS, TSS, and related output
Regression geometry	Chapters 7–8	Build design matrices, hat matrices, leverage values, rank checks, and residual degrees of freedom
MLR practice	Chapter 9	Fit full/reduced multiple-regression models, predict, compare fit, and plot residuals
Residual objects	Chapter 10	Extract fitted values, residuals, leverage, studentized residuals, coefficient covariance, and diagnostic plots
Transformations and encodings	Chapter 11	Fit polynomial, categorical, interaction, log-response, and linear-in-parameters models
Model repair	Chapter 12	Diagnose residuals, compare repaired models, and validate on a common response scale
Uncertainty workflows	Chapter 13	Extract coefficient intervals, mean-response intervals, prediction intervals, and fit metrics
Likelihood and GLMs	Chapter 14	Fit logistic and Poisson GLMs, read output, extract likelihoods, and compare nested models
Model selection	Chapter 15	Compare candidate models with AIC/BIC, cross-validation, subset or stepwise searches, and validation logs
Regularization and PCA/PCR	Chapter 16	Fit ridge, LASSO, elastic-net, PCA/PCR workflows with train-only preprocessing and validation
Neural-net benchmark	Chapter 17	Fit a compact neural-network regression benchmark and compare it with simpler baselines

2 Cookbook Operating Rules

Every recipe in this companion should be used under the same operating rules.

1. **Start from the analytic question.** A command is only useful if it answers a question that

matters.

2. **Declare the response and predictors.** Write down what each column means before modeling.
3. **Check variable types before summarizing or modeling.** A numeric-looking column may be a category code or an identifier.
4. **Clean before modeling, but document the cleaning.** Record what was dropped, recoded, imputed, or flagged.
5. **Split before learned preprocessing.** If using train/test validation, compute learned transformations from the training set only.
6. **Fit the simplest defensible model first.** More complicated repairs should be motivated by residual behavior, validation error, or domain knowledge.
7. **Extract diagnostics, not only coefficients.** Fitted values, residuals, leverage, and prediction error are part of the model output.
8. **Interpret before reporting.** Report what the output means for the original problem, including uncertainty and limitations.

3 Project Setup and Reproducibility

Purpose. Use a consistent project structure so the analysis can be rerun and audited.

A simple course-project folder can look like this:

```
project_name/
  README.md
  data/
    raw/
    cleaned/
  notebooks/
  scripts/
  outputs/
    figures/
    tables/
  reports/
```

Defensibility check. A reviewer should be able to answer: Where did the raw data come from? What cleaning decisions were made? Which file produces each figure or table?

Python: create common project folders

```
from pathlib import Path

project = Path("project_name")
for folder in [
```

```
    "data/raw",
    "data/cleaned",
    "notebooks",
    "scripts",
    "outputs/figures",
    "outputs/tables",
    "reports"
]:
    (project / folder).mkdir(parents=True, exist_ok=True)
```

R: create common project folders

```
folders <- c(
  "project_name/data/raw",
  "project_name/data/cleaned",
  "project_name/notebooks",
  "project_name/scripts",
  "project_name/outputs/figures",
  "project_name/outputs/tables",
  "project_name/reports"
)

for (folder in folders) {
  dir.create(folder, recursive = TRUE, showWarnings = FALSE)
}
```

README pattern

A short `README.md` should include:

```
# Project Name

## Question
What analytic question is this project answering?

## Data
Where did the raw data come from? When was it accessed?

## Workflow
1. Load raw data
2. Clean and validate variables
3. Run EDA
4. Fit models
5. Diagnose and compare models
6. Report conclusions

## Reproducibility
```

```
Which script or notebook reproduces the main results?
```

4 General Setup Patterns

Most Python examples use the following package pattern.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

Regression examples that require inference usually use `statsmodels`.

```
import statsmodels.api as sm
import statsmodels.formula.api as smf
```

Prediction-focused examples often use `sklearn`.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

A version-stable RMSE pattern is:

```
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
```

In R, most regression examples begin with `lm()`.

```
fit <- lm(y ~ x1 + x2, data = df)
summary(fit)
```

Common file imports

Use the file format actually supplied by the assignment or project. Keep the raw file unchanged and write cleaned versions to a separate location.

```
# Python
df_csv = pd.read_csv("data/raw/my_data.csv")
df_xlsx = pd.read_excel("data/raw/my_data.xlsx")
df_txt = pd.read_table("data/raw/bodyfat.txt", sep=r"\s+")

# R
df_csv <- read.csv("data/raw/my_data.csv")
df_txt <- read.table("data/raw/bodyfat.txt", header = TRUE)

library(readxl)
df_xlsx <- read_excel("data/raw/my_data.xlsx")
```

Older `.xls` files may require an additional engine/package or conversion to `.xlsx`. Do not change the data values while converting file formats.

5 Data Quality and Cleaning

5.1 Chapter 3 Support: Data-Quality First Pass

Purpose. Inspect a data table before summary statistics, plots, or models. This recipe answers: What columns exist, what types did the software infer, and where are the obvious missing or impossible values?

Python pattern

```
import pandas as pd

# Load raw data
raw_path = "data/raw/my_data.csv"
df = pd.read_csv(raw_path)

# Basic structure
print(df.shape)
print(df.head())
print(df.info())

# Column names and inferred types
print(df.columns.tolist())
print(df.dtypes)

# Missing counts and rates
missing_count = df.isna().sum()
missing_rate = df.isna().mean()
missing_summary = pd.DataFrame({
    "missing_count": missing_count,
    "missing_rate": missing_rate
}).sort_values("missing_rate", ascending=False)
print(missing_summary)

# Duplicate rows
print("duplicate rows:", df.duplicated().sum())
```

R pattern

```
# Load raw data
raw_path <- "data/raw/my_data.csv"
df <- read.csv(raw_path)

# Basic structure
dim(df)
head(df)
str(df)
```

```
# Missing counts and rates
missing_count <- colSums(is.na(df))
missing_rate <- colMeans(is.na(df))
missing_summary <- data.frame(
  missing_count = missing_count,
  missing_rate = missing_rate
)
missing_summary[order(-missing_summary$missing_rate), ]

# Duplicate rows
sum(duplicated(df))
```

Output to inspect. Row count, column count, column names, inferred data types, missingness rates, and duplicate rows.

Defensibility check. Can you explain whether each column is numeric, ordinal, categorical, date/time, text, identifier, or derived?

5.2 True Zeros, Placeholder Zeros, and Missingness Flags

Purpose. Distinguish valid zeros from placeholder zeros. A zero in `sibling_count` may be meaningful; a zero in `weight_lb` is usually invalid.

Python pattern

```
# Example: zero is not physically plausible for weight_lb
bad_weight_zero = df["weight_lb"] == 0
print("weight zero count:", bad_weight_zero.sum())

# Create a missingness flag before replacing values
df["weight_lb_was_zero"] = bad_weight_zero

# Replace placeholder zero with missing value
df.loc[bad_weight_zero, "weight_lb"] = np.nan

# Example: zero is valid for sibling_count
print(df["sibling_count"].value_counts(dropna=False).sort_index())
```

R pattern

```
# Example: zero is not physically plausible for weight_lb
bad_weight_zero <- df$weight_lb == 0
sum(bad_weight_zero, na.rm = TRUE)

# Create a missingness flag before replacing values
```

```
df$weight_lb_was_zero <- bad_weight_zero

# Replace placeholder zero with missing value
df$weight_lb[bad_weight_zero] <- NA

# Example: zero is valid for sibling_count
table(df$sibling_count, useNA = "ifany")
```

Common mistake. Do not run a universal "replace all zeros with missing" rule. The decision depends on the variable definition.

5.3 Binning and Grouped Summaries

Purpose. Create interpretable bins for a numerical variable and summarize another variable within those bins. This is useful after data cleaning, for example when placeholder zeros have been removed from variables such as body measurements.

Python pattern

```
# Keep only rows where both variables are usable
clean = df[(df["triceps"] > 0) & (df["bmi"] > 0)].copy()

clean["triceps_group"] = pd.cut(
    clean["triceps"],
    bins=[-np.inf, 20, 40, np.inf],
    right=False,
    labels=["under_20", "20_to_39", "40_plus"]
)

summary = (
    clean.groupby("triceps_group", observed=True)["bmi"]
         .agg(n="size", mean="mean", sd="std")
         .round(2)
)
print(summary)
```

R pattern

```
clean <- df[df$triceps > 0 & df$bmi > 0, ]

clean$triceps_group <- cut(
  clean$triceps,
  breaks = c(-Inf, 20, 40, Inf),
  right = FALSE,
  labels = c("under_20", "20_to_39", "40_plus")
)
```

```
n_by_group <- tapply(clean$bmi, clean$triceps_group, length)
mean_by_group <- tapply(clean$bmi, clean$triceps_group, mean)
sd_by_group <- tapply(clean$bmi, clean$triceps_group, sd)

summary_table <- cbind(
  n = n_by_group,
  mean = round(mean_by_group, 2),
  sd = round(sd_by_group, 2)
)
summary_table
```

Defensibility check. Bins should have a substantive interpretation. Avoid searching over many cutpoints until a pattern looks dramatic.

5.4 Dropping, Imputing, and Flagging Missing Values

Purpose. Implement the Chapter 3 decision workflow: drop only when justified, impute only when the assumption is defensible, and flag missingness when the fact of missingness may matter.

Python pattern

```
# Drop rows missing a required response variable
model_df = df.dropna(subset=["y"])

# Drop rows missing key predictors for a complete-case model
complete_df = model_df.dropna(subset=["x1", "x2"])

# Simple median imputation for a numerical predictor
x1_median = model_df["x1"].median()
model_df["x1_missing"] = model_df["x1"].isna()
model_df["x1_imputed"] = model_df["x1"].fillna(x1_median)

# Mode imputation for a categorical variable
mode_value = model_df["group"].mode(dropna=True).iloc[0]
model_df["group_missing"] = model_df["group"].isna()
model_df["group_imputed"] = model_df["group"].fillna(mode_value)
```

R pattern

```
# Drop rows missing a required response variable
model_df <- df[!is.na(df$y), ]

# Drop rows missing key predictors for a complete-case model
complete_df <- model_df[complete.cases(model_df[, c("x1", "x2")]), ]
```

```
# Simple median imputation for a numerical predictor
x1_median <- median(model_df$x1, na.rm = TRUE)
model_df$x1_missing <- is.na(model_df$x1)
model_df$x1_imputed <- ifelse(is.na(model_df$x1), x1_median, model_df$x1)

# Mode imputation for a categorical variable
mode_value <- names(sort(table(model_df$group), decreasing = TRUE))[1]
model_df$group_missing <- is.na(model_df$group)
model_df$group_imputed <- ifelse(
  is.na(model_df$group),
  mode_value,
  model_df$group
)
```

Output to inspect. Number of rows lost, missingness flags, imputed value used, and whether distributions changed after imputation.

Defensibility check. Be able to say: "This value was imputed because ..." or "This row was removed because ...". Imputation is an assumption, not a fact.

5.5 Fixing Data Types and Standardizing Columns

Purpose. Convert columns to types that match their analytical meaning and standardize names so later code is easier to audit.

Python pattern

```
# Standardize column names
clean_names = (
    df.columns
      .str.strip()
      .str.lower()
      .str.replace(" ", "_", regex=False)
      .str.replace("-", "_", regex=False)
)
df.columns = clean_names

# Convert numeric stored as text
df["height_in"] = pd.to_numeric(df["height_in"], errors="coerce")

# Convert categorical code to category, not numeric magnitude
df["service_branch"] = df["service_branch"].astype("category")

# Convert dates stored as text
df["date"] = pd.to_datetime(df["date"], errors="coerce")
```

R pattern

```
# Standardize column names
names(df) <- tolower(trimws(names(df)))
names(df) <- gsub(" ", "_", names(df))
names(df) <- gsub("-", "_", names(df))

# Convert numeric stored as text
df$height_in <- as.numeric(df$height_in)

# Convert categorical code to factor, not numeric magnitude
df$service_branch <- factor(df$service_branch)

# Convert dates stored as text
df$date <- as.Date(df$date)
```

Common mistake. Do not average category codes such as Army = 1, Marine Corps = 2, Navy = 3. Those are labels, not numerical amounts.

5.6 Impossible Values, Outlier Flags, and Cleaned Data

Purpose. Flag impossible values and save a cleaned data set with documented decisions.

Python pattern

```
# Flag impossible values
checks = {
    "height_nonpositive": df["height_in"] <= 0,
    "weight_nonpositive": df["weight_lb"] <= 0,
    "age_negative": df["age"] < 0
}

for name, mask in checks.items():
    print(name, mask.sum())

# Tukey outlier flag for a numeric column
q1 = df["score"].quantile(0.25)
q3 = df["score"].quantile(0.75)
iqr = q3 - q1
lower = q1 - 1.5 * iqr
upper = q3 + 1.5 * iqr

df["score_outlier_flag"] = (df["score"] < lower) | (df["score"] > upper)

# Save cleaned data
df.to_csv("data/cleaned/my_data_cleaned.csv", index=False)
```

R pattern

```
# Flag impossible values
sum(df$height_in <= 0, na.rm = TRUE)
sum(df$weight_lb <= 0, na.rm = TRUE)
sum(df$age < 0, na.rm = TRUE)

# Tukey outlier flag for a numeric column
q1 <- quantile(df$score, 0.25, na.rm = TRUE)
q3 <- quantile(df$score, 0.75, na.rm = TRUE)
iqr <- q3 - q1
lower <- q1 - 1.5 * iqr
upper <- q3 + 1.5 * iqr

df$score_outlier_flag <- df$score < lower | df$score > upper

# Save cleaned data
write.csv(df, "data/cleaned/my_data_cleaned.csv", row.names = FALSE)
```

Defensibility check. Outlier flags are not automatic deletion rules. If an observation is removed, record why.

6 EDA Summaries and Visuals

6.1 Chapter 4 Support: EDA Summaries and Visuals

Purpose. Compute summaries and plots that reveal center, spread, shape, extremes, and relationships among variables.

Python: summaries and basic plots

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Basic numerical summaries
df["score"].describe()

# Center and spread
mean_score = df["score"].mean()
median_score = df["score"].median()
sd_score = df["score"].std()
iqr_score = df["score"].quantile(0.75) - df["score"].quantile(0.25)

# Histogram with mean and median lines
ax = df["score"].hist(bins=20)
```

```
ax.axvline(mean_score, linestyle="--", label="mean")
ax.axvline(median_score, linestyle=":", label="median")
ax.legend()

# Grouped box plot
ax = df.boxplot(column="score", by="group")

# Correlation matrix
numeric_cols = ["score", "height_in", "weight_lb"]
corr = df[numeric_cols].corr()
```

R: summaries and basic plots

```
# Basic numerical summaries
summary(df$score)

# Center and spread
mean_score <- mean(df$score, na.rm = TRUE)
median_score <- median(df$score, na.rm = TRUE)
sd_score <- sd(df$score, na.rm = TRUE)
iqr_score <- IQR(df$score, na.rm = TRUE)

# Histogram with mean and median lines
hist(df$score)
abline(v = mean_score, lty = 2)
abline(v = median_score, lty = 3)

# Grouped box plot
boxplot(score ~ group, data = df)

# Correlation matrix
numeric_cols <- c("score", "height_in", "weight_lb")
cor(df[numeric_cols], use = "complete.obs")
```

Python: heatmap pattern

```
import seaborn as sns

corr = df[numeric_cols].corr()
sns.heatmap(corr, annot=True, center=0)
```

Python: seaborn plot variants

```
import seaborn as sns
import matplotlib.pyplot as plt

# Histogram, optionally on a log scale
```

```
sns.histplot(data=df, x="miles_per_capita", bins=25)
sns.histplot(data=df, x="miles_per_capita", bins=25, log_scale=True)

# Scatterplot with a horizontal reference line
ax = sns.scatterplot(data=df, x="population", y="miles_per_capita")
ax.axhline(22, color="black", linestyle="--")
plt.xscale("log")

# Pairwise scatterplot matrix
sns.pairplot(df[["y", "x1", "x2", "x3"]].dropna())
```

R: scatterplot matrix and correlation heatmap

```
numeric_cols <- c("y", "x1", "x2", "x3")

pairs(df[numeric_cols])

# Optional package for a labeled correlation heatmap
library(corrplot)
corr <- cor(df[numeric_cols], use = "complete.obs")
corrplot(corr, method = "color", addCoef.col = "black", tl.srt = 45)
```

R: optional `ggpairs` matrix

```
library(GGally)
library(ggplot2)

numeric_cols <- c("y", "x1", "x2", "x3")

ggpairs(
  df,
  columns = numeric_cols,
  upper = list(continuous = wrap("cor", size = 4)),
  lower = list(continuous = wrap("points", alpha = 0.4, size = 1)),
  diag = list(continuous = wrap("densityDiag", alpha = 0.6))
) +
  theme_bw(base_size = 12)
```

Output to inspect. Distributions, impossible values, skewness, unusually large spread, grouped differences, and strong pairwise correlations.

Defensibility check. A plot should answer a question. State what pattern the plot reveals and what follow-up it suggests.

7 Simple Linear Regression Workflow

7.1 Chapter 5 Support: Fitting Simple Linear Regression

Purpose. Fit an SLR model, extract coefficients, compute residuals, and create basic model-check plots.

Python: fit SLR with `statsmodels`

```
import pandas as pd
import statsmodels.api as sm

# df has columns: response y and predictor x
y = df["y"]
X = sm.add_constant(df["x"])  # adds intercept column

model = sm.OLS(y, X).fit()
print(model.summary())

beta0_hat = model.params["const"]
beta1_hat = model.params["x"]
residuals = model.resid
fitted = model.fittedvalues
rss = (residuals ** 2).sum()
mse = (residuals ** 2).mean()
```

Python: compute the least squares formulas manually

```
x = df["x"]
y = df["y"]

xbar = x.mean()
ybar = y.mean()

beta1_hat = ((x - xbar) * (y - ybar)).sum() / ((x - xbar) ** 2).sum()
beta0_hat = ybar - beta1_hat * xbar

print(beta0_hat, beta1_hat)
```

Python: quick scatterplot and residual plot

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(df["x"], df["y"])
ax.plot(df["x"], beta0_hat + beta1_hat * df["x"])
```

```
ax.set_xlabel("x")
ax.set_ylabel("y")
ax.set_title("Simple linear regression fit")

fig, ax = plt.subplots()
ax.scatter(fitted, residuals)
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals versus fitted values")
```

R: fit SLR with `lm`

```
# df has columns: y and x
fit <- lm(y ~ x, data = df)
summary(fit)

beta0_hat <- coef(fit)[1]
beta1_hat <- coef(fit)[2]
residuals <- resid(fit)
fitted_values <- fitted(fit)
rss <- sum(residuals^2)
mse <- mean(residuals^2)
```

R: manual formula

```
x <- df$x
y <- df$y

xbar <- mean(x)
ybar <- mean(y)

beta1_hat <- sum((x - xbar) * (y - ybar)) / sum((x - xbar)^2)
beta0_hat <- ybar - beta1_hat * xbar
```

Python: regression through the origin

```
import statsmodels.formula.api as smf
from sklearn.linear_model import LinearRegression

# statsmodels formula: remove the intercept with - 1
fit_origin = smf.ols("y ~ x - 1", data=df).fit()
print(fit_origin.summary())

# sklearn equivalent
X_origin = df[["x"]]
y_origin = df["y"]
```

```
rto = LinearRegression(fit_intercept=False).fit(X_origin, y_origin)
print(rto.coef_)

# Manual scalar formula
beta1_origin = (df["x"] * df["y"]).sum() / (df["x"] ** 2).sum()
```

R: regression through the origin

```
fit_origin <- lm(y ~ x + 0, data = df)
summary(fit_origin)

beta1_origin <- sum(df$x * df$y) / sum(df$x^2)
```

Regression-through-origin warning. Force the intercept to zero only when the scientific or operational setting requires the mean response to be zero at $x = 0$. Do not remove the intercept merely to improve a fit statistic.

Python: simulate regression assumptions

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import norm

np.random.seed(1)
x = np.linspace(-5, 5, 100)
mean_y = 21 - 3 * x

eps_constant = norm.rvs(loc=0, scale=4, size=len(x))
y_constant = mean_y + eps_constant

eps_increasing = np.abs(x) * norm.rvs(loc=0, scale=1, size=len(x))
y_increasing = mean_y + eps_increasing

plt.scatter(x, y_constant, label="constant variance")
plt.scatter(x, y_increasing, label="increasing variance")
plt.plot(x, mean_y, color="black", label="true mean")
plt.legend()
```

R: simulate regression assumptions

```
set.seed(1)
x <- seq(-5, 5, length.out = 100)
mean_y <- 21 - 3 * x

eps_constant <- rnorm(length(x), mean = 0, sd = 4)
y_constant <- mean_y + eps_constant
```

```
eps_increasing <- abs(x) * rnorm(length(x), mean = 0, sd = 1)
y_increasing <- mean_y + eps_increasing

plot(x, y_constant)
points(x, y_increasing, col = "red")
lines(x, mean_y, lwd = 2)
```

Common mistake. Coefficients are not automatically causal. They describe fitted associations unless the data collection and design justify a causal interpretation.

7.2 Chapter 6 Support: Fit Quality and Hypothesis Testing

Purpose. Extract fit quality and statistical-evidence quantities from an SLR model.

Python: SLR with `statsmodels`

```
import pandas as pd
import statsmodels.api as sm

# df has columns y and x
y = df["y"]
X = sm.add_constant(df["x"])

fit = sm.OLS(y, X).fit()
print(fit.summary())

r2 = fit.rsquared
f_stat = fit.fvalue
f_pvalue = fit.f_pvalue
residuals = fit.resid
fitted = fit.fittedvalues
rss = (residuals ** 2).sum()
tss = ((y - y.mean()) ** 2).sum()
manual_r2 = 1 - rss / tss

print(r2, f_stat, f_pvalue, manual_r2)
```

Python: manual R^2 and F

```
n = len(y)
rss = ((y - fitted) ** 2).sum()
tss = ((y - y.mean()) ** 2).sum()
r2 = 1 - rss / tss
F = (tss - rss) / (rss / (n - 2))
```

R: SLR with `lm`

```
fit <- lm(y ~ x, data = df)
summary(fit)

r2 <- summary(fit)$r.squared
f_stat <- summary(fit)$fstatistic
residuals <- resid(fit)
fitted_values <- fitted(fit)
rss <- sum(residuals^2)
tss <- sum((df$y - mean(df$y))^2)
manual_r2 <- 1 - rss / tss
```

R: manual F p-value

```
n <- nrow(df)
F_obs <- (tss - rss) / (rss / (n - 2))
p_value <- pf(F_obs, df1 = 1, df2 = n - 2, lower.tail = FALSE)
```

R: two-sample variance test

```
# Compare the variance of y across two groups
var.test(y ~ group, data = df)
```

Python: manual two-sample variance test

```
from scipy.stats import f

group_a = df.loc[df["group"] == "A", "y"].dropna()
group_b = df.loc[df["group"] == "B", "y"].dropna()

var_a = group_a.var(ddof=1)
var_b = group_b.var(ddof=1)

F_obs = var_a / var_b
df1 = len(group_a) - 1
df2 = len(group_b) - 1

p_lower = f.cdf(F_obs, df1, df2)
p_upper = 1 - f.cdf(F_obs, df1, df2)
p_value = 2 * min(p_lower, p_upper)
p_value = min(p_value, 1)

print(F_obs, p_value)
```

Variance-test caution. Classical variance tests are sensitive to distributional assumptions. Treat them as one piece of evidence alongside plots and the data context.

Output to inspect. R^2, RSS, TSS, slope estimate, slope p-value, overall F-statistic, and residual plot.

Defensibility check. Separate fit quality from statistical evidence. A high R^2 is not the same as proof of a useful or causal model.

8 Regression Geometry and Design Matrices

8.1 Chapter 7 Support: Design Matrix and Hat Matrix

Purpose. Build the design matrix, compute the hat matrix, and verify the projection identities used in the textbook.

Python: manual hat matrix

The direct inverse below is included to make the projection geometry visible. For routine modeling, prefer `statsmodels`, `lm()`, or a numerically stable least-squares solver rather than manually inverting $\mathbf{X}^T\mathbf{X}$.

```
import numpy as np

x = np.array([-1.0, 0.0, 1.0])
y = np.array([1.0, 0.0, 2.0])

X = np.column_stack([np.ones(len(x)), x])
beta_hat = np.linalg.inv(X.T @ X) @ X.T @ y
H = X @ np.linalg.inv(X.T @ X) @ X.T

y_hat = H @ y
e = y - y_hat

print(beta_hat)
print(y_hat)
print(e)
print(X.T @ e)       # should be close to zero
print(H.T - H)       # should be close to zero
print(H @ H - H)     # should be close to zero
print(np.allclose(H, H.T))
print(np.allclose(H @ H, H))
print(np.allclose(y_hat, X @ beta_hat))
```

Python: with `statsmodels`

```
import statsmodels.api as sm

X_sm = sm.add_constant(x)
fit = sm.OLS(y, X_sm).fit()

beta_hat = fit.params
y_hat = fit.fittedvalues
residuals = fit.resid

influence = fit.get_influence()
h_ii = influence.hat_matrix_diag
```

R: manual hat matrix

```
x <- c(-1, 0, 1)
y <- c(1, 0, 2)

X <- cbind(1, x)
beta_hat <- solve(t(X) %*% X) %*% t(X) %*% y
H <- X %*% solve(t(X) %*% X) %*% t(X)

y_hat <- H %*% y
e <- y - y_hat

t(X) %*% e      # should be zero
all.equal(H, t(H))
H %*% H - H     # should be zero
all.equal(H %*% H, H)
all.equal(as.vector(y_hat), as.vector(X %*% beta_hat))
```

R: with `lm`

```
fit <- lm(y ~ x)
coef(fit)
fitted(fit)
resid(fit)
hatvalues(fit)
```

Common mistake. Do not confuse projection in $\mathbb{R}^n$ observation space with drawing a line on the two-dimensional scatterplot. The fitted vector $\widehat{\mathbf{y}}$ is the projection object.

8.2 Chapter 8 Support: Degrees of Freedom and Identifiability

Purpose. Check rank, residual degrees of freedom, leverage sums, and non-identifiability.

Python: check rank and residual degrees of freedom

```
import numpy as np

# Example design matrix with intercept and two predictors
x1 = np.array([1, 2, 3, 4, 5], dtype=float)
x2 = np.array([2, 1, 3, 5, 4], dtype=float)
y = np.array([3, 4, 5, 7, 8], dtype=float)

X = np.column_stack([np.ones(len(x1)), x1, x2])
n, k = X.shape          # k = p + 1 if an intercept is included
r = np.linalg.matrix_rank(X)

print("n:", n)
print("columns k:", k)
print("rank:", r)
print("fitted-space df:", r)
print("residual df:", n - r)
print("full column rank?", r == k)
```

Python: detect a non-identifiable temperature design

```
import numpy as np

C = np.array([0, 10, 20, 30, 40], dtype=float)
F = 9/5 * C + 32

X = np.column_stack([np.ones(len(C)), C, F])

print(np.linalg.matrix_rank(X))  # rank is 2, but X has 3 columns
print(X @ np.array([-32, -9/5, 1.0]))  # should be zero
```

Python: null space from SVD

```
import numpy as np

U, s, Vt = np.linalg.svd(X)
rank = np.linalg.matrix_rank(X)
null_basis = Vt[rank:].T

print(null_basis)
```

Python: statsmodels residual degrees of freedom

```
import statsmodels.api as sm

X_sm = sm.add_constant(np.column_stack([x1, x2]))
```

```
fit = sm.OLS(y, X_sm).fit()

print(fit.df_model)   # predictor df beyond intercept
print(fit.df_resid)   # residual df
print(fit.get_influence().hat_matrix_diag.sum())  # trace(H)
```

R: rank and residual degrees of freedom

```
X <- cbind(1, x1, x2)
n <- nrow(X)
k <- ncol(X)
r <- qr(X)$rank

print(r)
print(n - r)
print(r == k)
```

R: regression output

```
fit <- lm(y ~ x1 + x2)
df.residual(fit)
length(coef(fit))
hatvalues(fit)
sum(hatvalues(fit))
```

Output to inspect. Matrix rank, number of columns, residual degrees of freedom, omitted or unstable coefficients, and leverage sum.

Defensibility check. If rank(X) is smaller than the number of columns, coefficient interpretation is not unique.

9 Multiple Linear Regression Workflow

9.1 Chapter 9 Support: Multiple Linear Regression in Practice

Purpose. Fit, interpret, predict, and compare multiple linear regression models using the Advertising example pattern.

Predictor-correlation first pass. Before interpreting individual MLR coefficients, inspect relationships among the predictors. Predictor-predictor correlations are useful early warnings for multicollinearity, but pairwise checks are incomplete because one predictor can be explained by several others together. Use the scatterplot-matrix pattern below and the Chapter 4 heatmap/pairplot recipes as first screens, then use rank or VIF checks when coefficient interpretation matters.

Python: scatterplot matrix

```
import pandas as pd
import matplotlib.pyplot as plt

ad = pd.read_csv("Advertising.csv")
cols = ["Sales", "TV", "Radio", "Newspaper"]
pd.plotting.scatter_matrix(ad[cols], figsize=(8, 8), diagonal="hist")
plt.tight_layout()
plt.show()
```

Python: full model with `statsmodels`

```
import statsmodels.api as sm

X = ad[["TV", "Radio", "Newspaper"]]
X = sm.add_constant(X)
y = ad["Sales"]

fit = sm.OLS(y, X).fit()
print(fit.summary())
```

Python: reduced model with TV and Radio

```
X_reduced = ad[["TV", "Radio"]]
X_reduced = sm.add_constant(X_reduced)
fit_reduced = sm.OLS(y, X_reduced).fit()

print(fit_reduced.summary())
```

Python: point prediction and intervals

```
new_market = pd.DataFrame({
    "const": [1.0],
    "TV": [100],
    "Radio": [20],
    "Newspaper": [30]
})

pred = fit.get_prediction(new_market)
print(pred.summary_frame(alpha=0.05))
```

The output includes the point prediction, a confidence interval for the mean response, and a prediction interval for a new observation.

Python: model fit values

```
rss = sum(fit.resid**2)
n = fit.nobs
p = len(fit.params) - 1
rse = (rss / (n - p - 1))**0.5

print("RSS:", rss)
print("RSE:", rse)
print("R^2:", fit.rsquared)
print("Adjusted R^2:", fit.rsquared_adj)
print("Residual df:", fit.df_resid)
```

Python: residual plot

```
import matplotlib.pyplot as plt

plt.scatter(fit.fittedvalues, fit.resid)
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.title("Residuals vs Fitted")
plt.show()
```

R: full model

```
ad <- read.csv("Advertising.csv")
fit <- lm(Sales ~ TV + Radio + Newspaper, data = ad)
summary(fit)
```

R: reduced model and prediction

```
fit.reduced <- lm(Sales ~ TV + Radio, data = ad)
summary(fit.reduced)

new.market <- data.frame(TV = 100, Radio = 20, Newspaper = 30)
predict(fit, new.market, interval = "confidence")
predict(fit, new.market, interval = "prediction")
```

Python: candidate model comparison

```
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
```

```
# model_df contains y and vetted predictors only.
train, test = train_test_split(model_df, test_size=0.2, random_state=1701)

candidate_sets = {
    "small": ["waist"],
    "medium": ["waist", "chest", "hip", "weight"],
    "large": [c for c in model_df.columns if c != "y"]
}

rows = []
for name, cols in candidate_sets.items():
    fit = LinearRegression().fit(train[cols], train["y"])
    train_pred = fit.predict(train[cols])
    test_pred = fit.predict(test[cols])
    rows.append({
        "model": name,
        "predictors": ", ".join(cols),
        "train_rmse": np.sqrt(mean_squared_error(train["y"], train_pred)),
        "test_rmse": np.sqrt(mean_squared_error(test["y"], test_pred))
    })

results = pd.DataFrame(rows).sort_values("test_rmse")
print(results)
```

R: candidate model comparison

```
# model_df contains y and vetted predictors only.
set.seed(1701)
idx <- sample(seq_len(nrow(model_df)), size = floor(0.8 * nrow(model_df)))
train <- model_df[idx, ]
test <- model_df[-idx, ]

fit_small <- lm(y ~ waist, data = train)
fit_medium <- lm(y ~ waist + chest + hip + weight, data = train)
fit_large <- lm(y ~ ., data = train)

train_rmse <- function(fit) {
  sqrt(mean(resid(fit)^2))
}

test_rmse <- function(fit) {
  pred <- predict(fit, newdata = test)
  sqrt(mean((test$y - pred)^2))
}

results <- data.frame(
```

```
  model = c("small", "medium", "large"),
  train_rmse = c(
    train_rmse(fit_small),
    train_rmse(fit_medium),
    train_rmse(fit_large)
  ),
  test_rmse = c(
    test_rmse(fit_small),
    test_rmse(fit_medium),
    test_rmse(fit_large)
  )
)
results[order(results$test_rmse), ]
```

Use `y ~ .` only after removing identifiers, forbidden predictors, variables unavailable at prediction time, and variables derived from the response.

Defensibility check. Interpret each coefficient as an adjusted association: the expected change in the response for a one-unit change in that predictor, holding the other included predictors fixed.

10 Estimator Behavior and Residual Objects

10.1 Chapter 10 Support: Python Residual and Coefficient Objects

Purpose. Extract the objects used in the truth-plus-error and residual-geometry chapters.

Fit with `statsmodels`

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# y is the response Series.
# X_raw is a DataFrame containing predictor columns only.
X = sm.add_constant(X_raw)       # adds intercept column
fit = sm.OLS(y, X).fit()

print(fit.summary())
```

Extract coefficients, fitted values, and residuals

```
beta_hat = fit.params
fitted = fit.fittedvalues
resid = fit.resid
sigma_hat = np.sqrt(fit.scale)   # residual standard error
```

Build the hat matrix directly

For moderate-sized datasets, the hat matrix can be computed directly for diagnostic or instructional purposes. For routine coefficient estimation, rely on the fitted-model object or a stable solver rather than explicitly inverting $\mathbf{X}^T\mathbf{X}$.

```
Xmat = X.to_numpy()
H = Xmat @ np.linalg.inv(Xmat.T @ Xmat) @ Xmat.T
M = np.eye(Xmat.shape[0]) - H
```

For large datasets, do not build the full $n \times n$ hat matrix unless needed. The diagonal leverage values are often enough:

```
influence = fit.get_influence()
leverage = influence.hat_matrix_diag
student_resid = influence.resid_studentized_internal
```

Residual plot

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.scatter(fitted, resid)
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs fitted values")
plt.show()
```

Q-Q plot

```
import statsmodels.api as sm

sm.qqplot(resid, line="45")
plt.show()
```

Coefficient covariance matrix

```
coef_cov = fit.cov_params()
standard_errors = fit.bse
```

These implement the estimated version of

$$\mathrm{Var}(\widehat{\boldsymbol{\beta}} \mid \mathbf{X}) = \sigma^2(\mathbf{X}^T\mathbf{X})^{-1}. \tag{.1}$$

10.2 Chapter 10 Support: R Residual and Coefficient Objects

Fit the model

```
fit <- lm(Y ~ X1 + X2 + X3, data = df)
summary(fit)
```

Extract fitted values and residuals

```
beta_hat <- coef(fit)
fitted_values <- fitted(fit)
residuals <- resid(fit)
sigma_hat <- summary(fit)$sigma
```

Hat values and studentized residuals

```
h <- hatvalues(fit)
r_student <- rstudent(fit)
r_standard <- rstandard(fit)
```

Diagnostic plots

```
plot(fit)
```

The default diagnostic plots include residuals versus fitted values, Q-Q plot, scale-location plot, and residuals versus leverage.

Output to inspect. Coefficients, standard errors, residual standard error, residual plots, leverage values, and studentized residuals.

11 Transformations and Linear-in-Parameters Models

11.1 Chapter 11 Support: Transformation Domain Checks

Purpose. Check whether a transformation is legal before fitting the model. Transformations repair a modeling problem only when the transformed quantity is meaningful for the data.

```
# Python checks
if (df["x"] <= 0).any():
    raise ValueError("log(x) requires x > 0")

if (df["y"] <= 0).any():
    raise ValueError("log(y) requires y > 0")

if (df["y"] < 0).any():
    raise ValueError("sqrt(y) requires y >= 0")
```

```
# R checks
stopifnot(all(df$x > 0, na.rm = TRUE))   # log(x)
stopifnot(all(df$y > 0, na.rm = TRUE))   # log(y)
stopifnot(all(df$y >= 0, na.rm = TRUE))  # sqrt(y)
```

Common mistake. Do not use `abs()` on real data merely to make `log()` or `sqrt()` legal. If a value has the wrong sign for a transformation, understand the data-generating context first.

11.2 Chapter 11 Support: Python Transformations

Purpose. Change the design columns while keeping the linear-in-parameters least-squares machinery.

Fit polynomial regression with `statsmodels`

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# df has columns y and x
x = df["x"]
X = pd.DataFrame({
    "x": x,
    "x2": x**2,
    "x3": x**3
})
X = sm.add_constant(X)

fit = sm.OLS(df["y"], X).fit()
print(fit.summary())
```

Center before polynomial terms

```
u = df["x"] - df["x"].mean()
X = pd.DataFrame({
    "u": u,
    "u2": u**2,
    "u3": u**3
})
X = sm.add_constant(X)
fit = sm.OLS(df["y"], X).fit()
```

Interaction model

```
X = pd.DataFrame({
    "x1": df["x1"],
```

```
    "x2": df["x2"],
    "x1_x2": df["x1"] * df["x2"]
})
X = sm.add_constant(X)
fit = sm.OLS(df["y"], X).fit()
```

Formula interface

```
import statsmodels.formula.api as smf

fit_poly = smf.ols("y ~ x + I(x**2) + I(x**3)", data=df).fit()
fit_int = smf.ols("y ~ x1 * x2", data=df).fit()
fit_log = smf.ols("np.log(y) ~ x", data=df).fit()
```

In the formula `y ~ x1 * x2`, the star expands to main effects plus interaction:

$$x_1 * x_2 \quad \Longrightarrow \quad x_1 + x_2 + x_1 x_2. \tag{.2}$$

Categorical predictors with formulas

```
import statsmodels.formula.api as smf

# curriculum is categorical; Other is the reference level.
fit_cat = smf.ols(
    "iq ~ C(curriculum, Treatment(reference=\"Other\"))",
    data=df
).fit()
print(fit_cat.summary())
```

Numeric-by-categorical interaction

```
fit_group_slope = smf.ols(
    "iq ~ hours * C(curriculum, Treatment(reference=\"Other\"))",
    data=df
).fit()
print(fit_group_slope.summary())
```

In formula syntax, `hours * C(curriculum)` expands to main effects plus the interaction. Use the fitted output to interpret reference-category intercepts, category contrasts, and group-specific slope changes.

Inspect dummy columns manually

```
# Put Other first so drop_first=True drops the reference category.
df["curriculum"] = pd.Categorical(
    df["curriculum"],
```

```
    categories=["Other", "OR", "Physics", "Applied Math", "Engineering"]
)
dummies = pd.get_dummies(df["curriculum"], drop_first=True, dtype=float)
X = pd.concat([df[["hours"]], dummies], axis=1)
X = sm.add_constant(X)
fit_manual = sm.OLS(df["iq"], X).fit()
```

Manual dummy creation is useful for inspecting the design matrix. For routine modeling, formulas are usually clearer.

Check encoded-design rank

```
import numpy as np

rank = np.linalg.matrix_rank(X.to_numpy())
n_cols = X.shape[1]

print("rank:", rank)
print("number of columns:", n_cols)
print("full column rank?", rank == n_cols)
```

If the rank is smaller than the number of columns, at least one design column is exactly redundant and the coefficient vector is not uniquely identified. Common causes include an intercept plus all dummy columns, duplicated variables, totals included with their components, and sparse interaction categories.

Prediction on a grid

```
x_grid = np.linspace(df["x"].min(), df["x"].max(), 200)
X_grid = pd.DataFrame({
    "const": 1.0,
    "x": x_grid,
    "x2": x_grid**2,
    "x3": x_grid**3
})
y_hat = fit.predict(X_grid)
```

Python: transformation simulation

```
import numpy as np
import matplotlib.pyplot as plt
import statsmodels.api as sm

def transformation_simulation(X, f, g, g_inv, eps_sd):
    eps = np.random.normal(loc=0, scale=eps_sd, size=len(X))
    fX = f(X)
```

```
    y = g_inv(3 + 4 * fX + eps)

    X_orig = sm.add_constant(X)
    fit_orig = sm.OLS(y, X_orig).fit()

    gy = g(y)
    X_trans = sm.add_constant(fX)
    fit_trans = sm.OLS(gy, X_trans).fit()

    fig, axes = plt.subplots(2, 2, figsize=(9, 7))

    order = np.argsort(X)
    axes[0, 0].scatter(X, y)
    axes[0, 0].plot(X[order], fit_orig.predict(X_orig)[order], color="red")
    axes[0, 0].set_title("Original scale")

    axes[1, 0].scatter(X, fit_orig.resid)
    axes[1, 0].axhline(0, color="red")
    axes[1, 0].set_title("Original residuals")

    order_t = np.argsort(fX)
    axes[0, 1].scatter(fX, gy)
    axes[0, 1].plot(
        fX[order_t],
        fit_trans.predict(X_trans)[order_t],
        color="red"
    )
    axes[0, 1].set_title("Transformed scale")

    axes[1, 1].scatter(fX, fit_trans.resid)
    axes[1, 1].axhline(0, color="red")
    axes[1, 1].set_title("Transformed residuals")

    plt.tight_layout()
    return fit_orig, fit_trans

np.random.seed(1)
X_quad = np.random.normal(loc=2, scale=4, size=100)
transformation_simulation(X_quad, lambda x: x**2, lambda y: y,
                          lambda z: z, eps_sd=75)

X_log_y = np.random.normal(loc=50, scale=16, size=100)
transformation_simulation(X_log_y, lambda x: x, np.log,
                          np.exp, eps_sd=3)
```

11.3 Chapter 11 Support: R Transformations

Fit polynomial regression

```
fit_poly <- lm(y ~ x + I(x^2) + I(x^3), data = df)
summary(fit_poly)
```

Use orthogonal polynomials

```
fit_orth <- lm(y ~ poly(x, degree = 3), data = df)
summary(fit_orth)
```

Use raw polynomial columns

```
fit_raw <- lm(y ~ poly(x, degree = 3, raw = TRUE), data = df)
summary(fit_raw)
```

Interaction model

```
fit_int <- lm(y ~ x1 * x2, data = df)
summary(fit_int)
```

This expands to:

```
y ~ x1 + x2 + x1:x2
```

Categorical predictors and reference levels

```
df$curriculum <- as.factor(df$curriculum)
df$curriculum <- relevel(df$curriculum, ref = "Other")

fit_cat <- lm(iq ~ curriculum, data = df)
summary(fit_cat)
```

Mixed numeric/categorical model and interaction

```
fit_mixed <- lm(iq ~ hours + curriculum, data = df)
fit_int   <- lm(iq ~ hours * curriculum, data = df)

summary(fit_mixed)
summary(fit_int)
```

Inspect the encoded design matrix

```
X <- model.matrix(~ hours * curriculum, data = df)
```

```
head(X)
rank <- qr(X)$rank
n_cols <- ncol(X)
rank
n_cols
rank == n_cols
```

Rank smaller than the number of columns means exact redundancy in the encoded design. For factors and interactions, inspect the model matrix before interpreting missing, unstable, or aliased coefficients.

Transformed response

```
fit_log_y <- lm(log(y) ~ x1 + x2, data = df)
summary(fit_log_y)
```

R: transformation simulation

```
transformation_simulation <- function(X, f, g, g_inv, eps_sd) {
  eps <- rnorm(length(X), mean = 0, sd = eps_sd)
  fX <- f(X)
  y <- g_inv(3 + 4 * fX + eps)

  fit_orig <- lm(y ~ X)
  gy <- g(y)
  fit_trans <- lm(gy ~ fX)

  old_par <- par(mfrow = c(2, 2))
  on.exit(par(old_par))

  plot(X, y, xlab = "Original predictor", ylab = "Original response")
  abline(fit_orig, col = "red")

  plot(X, resid(fit_orig), xlab = "Original predictor",
       ylab = "Original residuals")
  abline(h = 0, col = "red")

  plot(fX, gy, xlab = "Transformed predictor",
       ylab = "Transformed response")
  abline(fit_trans, col = "red")

  plot(fX, resid(fit_trans), xlab = "Transformed predictor",
       ylab = "Transformed residuals")
  abline(h = 0, col = "red")

  list(original = fit_orig, transformed = fit_trans)
}
```

```
set.seed(1)
X_quad <- rnorm(100, mean = 2, sd = 4)
transformation_simulation(X_quad, function(x) x^2, identity,
                          identity, eps_sd = 75)

X_log_y <- rnorm(100, mean = 50, sd = 16)
transformation_simulation(X_log_y, identity, log, exp, eps_sd = 3)
```

Defensibility check. After adding transformed columns, compare residual behavior and validation error. A more complicated design matrix must improve the analysis, not only the training fit.

12 Residual Diagnostics and Model Repair

12.1 Chapter 12 Support: Python Model Diagnostics and Repair

Purpose. Diagnose leftover pattern, try targeted repairs, and compare candidate models on a common scale.

Fit a baseline model and extract residuals

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt

# df has columns y, x1, x2
X = df[["x1", "x2"]]
X = sm.add_constant(X)
y = df["y"]

fit = sm.OLS(y, X).fit()
df["fitted"] = fit.fittedvalues
df["resid"] = fit.resid
print(fit.summary())
```

Residuals versus fitted values

```
fig, ax = plt.subplots()
ax.scatter(df["fitted"], df["resid"])
ax.axhline(0, linestyle="--")
ax.set_xlabel("Fitted values")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs fitted values")
```

Residuals versus a predictor

```
fig, ax = plt.subplots()
ax.scatter(df["x1"], df["resid"])
ax.axhline(0, linestyle="--")
ax.set_xlabel("x1")
ax.set_ylabel("Residuals")
ax.set_title("Residuals vs x1")
```

Partial residual plots

Partial residual plots examine a predictor's adjusted relationship with the response after accounting for the other predictors in the model. They are not the same as raw residuals versus one predictor.

```
import statsmodels.graphics.api as smg

fig = smg.plot_partregress_grid(fit)
```

Normal Q-Q plot

```
fig = sm.qqplot(fit.resid, line="45")
plt.title("Normal Q-Q plot of residuals")
```

Scale-location plot

```
infl = fit.get_influence()
std_resid = infl.resid_studentized_internal

fig, ax = plt.subplots()
ax.scatter(fit.fittedvalues, np.sqrt(np.abs(std_resid)))
ax.set_xlabel("Fitted values")
ax.set_ylabel("Sqrt(|studentized residuals|)")
ax.set_title("Scale-location plot")
```

Breusch-Pagan test

```
from statsmodels.stats.diagnostic import het_breuschpagan

bp_stat, bp_pvalue, f_stat, f_pvalue = het_breuschpagan(
    fit.resid,
    fit.model.exog
)
print(bp_stat, bp_pvalue, f_stat, f_pvalue)
```

Use the BP test as supporting evidence for nonconstant variance. Inspect the residual plots before deciding how to repair the model.

Variance inflation factors

```
import pandas as pd
import statsmodels.api as sm
from statsmodels.stats.outliers_influence import variance_inflation_factor

ad = pd.read_csv("Advertising.csv")
predictors = ["TV", "Radio", "Newspaper"]

X_vif = ad[predictors].dropna()
X_vif = sm.add_constant(X_vif)

vif_table = pd.DataFrame({
    "term": X_vif.columns,
    "VIF": [
        variance_inflation_factor(X_vif.to_numpy(), i)
        for i in range(X_vif.shape[1])
    ]
})

# Do not interpret the constant/intercept VIF.
vif_table = vif_table[
    ~vif_table["term"].str.lower().isin(["const", "intercept"])
]
print(vif_table.round(3))
```

VIF near 1 gives little evidence of variance inflation. Values above 5 or 10 are screening warnings, not automatic deletion rules. Large VIFs mainly warn about unstable coefficients and inflated standard errors; prediction can still be acceptable if validation error is good.

Condition number screen

```
import numpy as np
import pandas as pd

ad = pd.read_csv("Advertising.csv")
predictors = ["TV", "Radio", "Newspaper"]
X_num = ad[predictors].dropna()

# Remove zero-variance columns before standardizing.
nonconstant = X_num.columns[X_num.nunique(dropna=True) > 1]
X_num = X_num[nonconstant]

X_scaled = (X_num - X_num.mean()) / X_num.std(ddof=0)

print("rank:", np.linalg.matrix_rank(X_scaled.to_numpy()))
print("number of columns:", X_scaled.shape[1])
```

```
print("condition number:", np.linalg.cond(X_scaled.to_numpy()))
```

Compute condition-number screens on predictor columns, not on the intercept. Scaling keeps units from dominating the diagnostic, and zero-variance columns must be removed before scaling.

Influence and leverage

```
infl = fit.get_influence()
df["leverage"] = infl.hat_matrix_diag
df["student_resid"] = infl.resid_studentized_external
df["cooks_d"] = infl.cooks_distance[0]

avg_leverage = len(fit.params) / fit.nobs
high_lev_2x = 2 * avg_leverage
high_lev_3x = 3 * avg_leverage

print("average leverage:", avg_leverage)
print("2x, 3x flags:", high_lev_2x, high_lev_3x)
print(df.sort_values("cooks_d", ascending=False).head())
```

Try a polynomial repair

```
df["x1_c"] = df["x1"] - df["x1"].mean()
df["x1_c2"] = df["x1_c"] ** 2

X_poly = df[["x1_c", "x1_c2", "x2"]]
X_poly = sm.add_constant(X_poly)
fit_poly = sm.OLS(y, X_poly).fit()
print(fit_poly.summary())
```

Try an interaction repair

```
df["x1_x2"] = df["x1"] * df["x2"]
X_int = df[["x1", "x2", "x1_x2"]]
X_int = sm.add_constant(X_int)
fit_int = sm.OLS(y, X_int).fit()
print(fit_int.summary())
```

Try a response transformation

```
# Requires y > 0
fit_log = sm.OLS(np.log(y), X).fit()

# Predictions returned to original scale
log_pred = fit_log.predict(X)
y_pred = np.exp(log_pred)
```

Compare using validation RMSE on the same scale

When comparing repaired models, compute validation error on the response scale that matters for the analysis.

```
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

train, test = train_test_split(df, test_size=0.25, random_state=1)

# Baseline model
X_train = sm.add_constant(train[["x1", "x2"]])
X_test = sm.add_constant(test[["x1", "x2"]])
fit_base = sm.OLS(train["y"], X_train).fit()
pred_base = fit_base.predict(X_test)

# Polynomial repair: center using the training mean only
x1_mean = train["x1"].mean()
train = train.copy()
test = test.copy()
train["x1_c"] = train["x1"] - x1_mean
test["x1_c"] = test["x1"] - x1_mean
train["x1_c2"] = train["x1_c"] ** 2
test["x1_c2"] = test["x1_c"] ** 2

X_train_poly = sm.add_constant(train[["x1_c", "x1_c2", "x2"]])
X_test_poly = sm.add_constant(test[["x1_c", "x1_c2", "x2"]])
fit_poly = sm.OLS(train["y"], X_train_poly).fit()
pred_poly = fit_poly.predict(X_test_poly)

rmse_base = np.sqrt(mean_squared_error(test["y"], pred_base))
rmse_poly = np.sqrt(mean_squared_error(test["y"], pred_poly))
print(rmse_base, rmse_poly)
```

12.2 Chapter 12 Support: R Model Diagnostics and Repair

Fit, plot residuals, and inspect diagnostics

```
fit <- lm(y ~ x1 + x2, data = df)
summary(fit)

plot(fitted(fit), resid(fit))
abline(h = 0, lty = 2)

qqnorm(resid(fit))
qqline(resid(fit))

plot(fit)   # standard diagnostic panel
```

Select individual R diagnostic plots

```
plot(fit, which = 1)  # Residuals vs fitted
plot(fit, which = 2)  # Normal Q-Q
plot(fit, which = 3)  # Scale-location
plot(fit, which = 5)  # Residuals vs leverage with Cook distance
```

Partial residual plots

```
termplot(fit, partial.resid = TRUE)
```

Breusch-Pagan test

```
library(lmtest)
bptest(fit)
```

Use this as supporting evidence for heteroskedasticity, not as a replacement for the residual and scale-location plots.

Variance inflation factors

```
library(car)

ad <- read.csv("Advertising.csv")
fit_ad <- lm(Sales ~ TV + Radio + Newspaper, data = ad)
car::vif(fit_ad)
```

Do not interpret an intercept or constant VIF. For models with factors, VIF tools may report grouped or generalized VIF output; inspect the encoded design matrix before applying numeric-predictor thresholds mechanically.

Condition number screen

```
predictors <- c("TV", "Radio", "Newspaper")
ad <- read.csv("Advertising.csv")
X_num <- na.omit(ad[predictors])

# Remove constant columns before scale().
keep <- sapply(X_num, function(z) length(unique(z)) > 1)
X_num <- X_num[, keep, drop = FALSE]
X_scaled <- scale(X_num)

qr(X_scaled)$rank
ncol(X_scaled)
kappa(X_scaled)
```

Small rank relative to the number of columns signals exact redundancy. A large condition number is a near-collinearity warning for coefficient interpretation, not an automatic reason to delete variables.

Polynomial and interaction repairs

```
fit_poly <- lm(y ~ x1 + I(x1^2) + x2, data = df)
fit_int  <- lm(y ~ x1 * x2, data = df)

anova(fit, fit_poly)
anova(fit, fit_int)
```

Response transformation

```
fit_log <- lm(log(y) ~ x1 + x2, data = df)
summary(fit_log)
```

Influence

```
h <- hatvalues(fit)
r_student <- rstudent(fit)
D <- cooks.distance(fit)

avg_h <- length(coef(fit)) / nobs(fit)
which(h > 2 * avg_h)
which(h > 3 * avg_h)
sort(D, decreasing = TRUE)[1:5]
```

Documenting multicollinearity decisions. Record which predictors were collinear, whether the issue was exact rank deficiency or near-collinearity, whether the goal was prediction or interpretation/inference, what repair was attempted, and whether validation error or residual behavior changed.

Defensibility check. A repair is justified when it addresses a visible diagnostic issue and improves the analysis without hiding an important limitation. For multicollinearity, simplifying or combining predictors is usually most important when coefficient interpretation or inference is central; for prediction, keeping redundant predictors can be defensible if validation performance is good and the coefficients are not over-interpreted.

13 Confidence and Prediction Interval Workflows

Purpose. Support Chapter 13 by extracting coefficient confidence intervals, mean-response intervals, prediction intervals, and basic prediction-error summaries. Use these recipes after fitting a defensible linear model and checking that the assumptions behind interval statements are plausible enough for the analysis.

Python: coefficient intervals and prediction intervals

```
import numpy as np
import pandas as pd
import statsmodels.api as sm

# X_raw contains predictor columns only; y is the response.
X = sm.add_constant(X_raw)
fit = sm.OLS(y, X).fit()

# Coefficient confidence intervals
coef_table = pd.DataFrame({
    "estimate": fit.params,
    "std_error": fit.bse,
    "ci_low": fit.conf_int(alpha=0.05)[0],
    "ci_high": fit.conf_int(alpha=0.05)[1]
})
print(coef_table)

# Mean-response and prediction intervals for new rows
new_X = pd.DataFrame({
    "const": [1.0],
    "x1": [10.0],
    "x2": [5.0]
})
pred = fit.get_prediction(new_X)
print(pred.summary_frame(alpha=0.05))
```

In the prediction summary, mean-interval columns describe uncertainty in the fitted mean response. Observation-interval columns describe uncertainty for a new individual response and should usually be wider.

R: coefficient, mean-response, and prediction intervals

```
fit <- lm(y ~ x1 + x2, data = df)

# Coefficient confidence intervals
confint(fit, level = 0.95)

# Intervals at new predictor settings
new_df <- data.frame(x1 = 10, x2 = 5)
predict(fit, newdata = new_df, interval = "confidence", level = 0.95)
predict(fit, newdata = new_df, interval = "prediction", level = 0.95)
```

Fit and prediction-error summaries

```
# Python
```

```
resid = fit.resid
rmse = np.sqrt(np.mean(resid**2))
mae = np.mean(np.abs(resid))
print({
    "RMSE": rmse,
    "MAE_or_MAD": mae,
    "R2": fit.rsquared,
    "Adjusted_R2": fit.rsquared_adj
})
```

```
# R
resid_vals <- resid(fit)
rmse <- sqrt(mean(resid_vals^2))
mae <- mean(abs(resid_vals))
summary_fit <- summary(fit)

list(
  RMSE = rmse,
  MAE_or_MAD = mae,
  R2 = summary_fit$r.squared,
  Adjusted_R2 = summary_fit$adj.r.squared
)
```

Optional: LAD / median-regression pattern

Use LAD only when absolute-error loss is substantively meaningful. Treat it as an optional alternative, not a replacement for checking assumptions.

```
# Python: median regression as an LAD-style fit
import statsmodels.formula.api as smf
lad_fit = smf.quantreg("y ~ x1 + x2", data=df).fit(q=0.5)
print(lad_fit.summary())
```

```
# R: optional package pattern
# install.packages("quantreg") if needed
library(quantreg)
lad_fit <- rq(y ~ x1 + x2, data = df, tau = 0.5)
summary(lad_fit)
```

Defensibility check. State which interval you are reporting: coefficient interval, mean-response interval, or prediction interval. Do not describe a confidence interval for the mean response as if it were a prediction interval for a future individual.

14 Likelihood, GLMs, and Model Tests

Purpose. Support Chapter 14 with compact workflows for logistic regression, Poisson regression, likelihood extraction, and nested-model comparison. These recipes are for implementation and output reading; the probability model and link-function reasoning belong in the main textbook.

Python: logistic regression for a binary response

```
import statsmodels.formula.api as smf

# y_binary should be coded 0/1.
fit_logit = smf.logit("y_binary ~ x1 + x2", data=df).fit()
print(fit_logit.summary())

# Predicted probabilities
pred_prob = fit_logit.predict(df[["x1", "x2"]])
print(pred_prob.head())
```

R: logistic regression

```
fit_logit <- glm(y_binary ~ x1 + x2, data = df, family = binomial())
summary(fit_logit)

# Predicted probabilities
p_hat <- predict(fit_logit, type = "response")
head(p_hat)
```

Python: Poisson regression for count responses

```
import statsmodels.api as sm
import statsmodels.formula.api as smf

fit_pois = smf.glm(
    "count_y ~ x1 + x2",
    data=df,
    family=sm.families.Poisson()
).fit()
print(fit_pois.summary())

# Predicted mean counts
mu_hat = fit_pois.predict(df[["x1", "x2"]])
print(mu_hat.head())
```

R: Poisson regression

```
fit_pois <- glm(count_y ~ x1 + x2, data = df, family = poisson())
```

```
summary(fit_pois)

# Predicted mean counts
mu_hat <- predict(fit_pois, type = "response")
head(mu_hat)
```

Likelihood values and nested-model comparison

```
# Python: likelihood-ratio comparison for nested models
from scipy import stats

full = smf.logit("y_binary ~ x1 + x2 + x3", data=df).fit()
reduced = smf.logit("y_binary ~ x1 + x2", data=df).fit()

lr_stat = 2 * (full.llf - reduced.llf)
df_diff = int(full.df_model - reduced.df_model)
p_value = stats.chi2.sf(lr_stat, df_diff)

print({"LR_stat": lr_stat, "df": df_diff, "p_value": p_value})
print({"AIC_full": full.aic, "AIC_reduced": reduced.aic})

# R: nested-model comparison
full <- glm(y_binary ~ x1 + x2 + x3, data = df, family = binomial())
reduced <- glm(y_binary ~ x1 + x2, data = df, family = binomial())

anova(reduced, full, test = "Chisq")
logLik(full)
AIC(full, reduced)
BIC(full, reduced)
```

Output to inspect. Coefficient estimates, standard errors, link-function scale, predicted probabilities or mean counts, log-likelihood, AIC/BIC, and whether model comparisons are nested when using likelihood-ratio tests.

Defensibility check. Logistic regression and Poisson regression are not ordinary linear regression with a different response name. Check that the response type and link function match the modeling question.

15 Model Selection and Cross-Validation

Purpose. Support Chapter 15 by creating compact model-comparison tables and validation workflows. These recipes help document how a model was selected; they do not make the selected model automatically true or immune from diagnostics.

Python: AIC/BIC comparison table

```
import pandas as pd
import statsmodels.formula.api as smf

formulas = {
    "small": "y ~ x1 + x2",
    "medium": "y ~ x1 + x2 + x3",
    "large": "y ~ x1 + x2 + x3 + x4"
}

rows = []
fits = {}
for name, formula in formulas.items():
    fit = smf.ols(formula, data=df).fit()
    fits[name] = fit
    rows.append({
        "model": name,
        "formula": formula,
        "AIC": fit.aic,
        "BIC": fit.bic,
        "adj_R2": fit.rsquared_adj,
        "RMSE_train": ((fit.resid ** 2).mean() ** 0.5)
    })

compare = pd.DataFrame(rows).sort_values("AIC")
print(compare)
```

R: AIC/BIC comparison table

```
fit_small <- lm(y ~ x1 + x2, data = df)
fit_medium <- lm(y ~ x1 + x2 + x3, data = df)
fit_large <- lm(y ~ x1 + x2 + x3 + x4, data = df)

AIC(fit_small, fit_medium, fit_large)
BIC(fit_small, fit_medium, fit_large)
```

Python: k-fold CV and LOOCV templates

```
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import KFold, LeaveOneOut, cross_val_score
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = df[["x1", "x2", "x3"]]
y = df["y"]
```

```
pipe = make_pipeline(StandardScaler(), LinearRegression())

kf = KFold(n_splits=5, shuffle=True, random_state=1701)
rmse_scores = -cross_val_score(
    pipe, X, y, cv=kf, scoring="neg_root_mean_squared_error"
)
print("5-fold RMSE:", rmse_scores.mean(), rmse_scores.std())

# LOOCV can be slow for large data.
loo = LeaveOneOut()
loo_rmse = -cross_val_score(
    pipe, X, y, cv=loo, scoring="neg_root_mean_squared_error"
)
print("LOOCV RMSE:", loo_rmse.mean())
```

R: stepwise search with validation afterward

```
set.seed(1701)
idx <- sample(seq_len(nrow(df)), size = floor(0.75 * nrow(df)))
train <- df[idx, ]
test <- df[-idx, ]

base <- lm(y ~ 1, data = train)
full <- lm(y ~ x1 + x2 + x3 + x4, data = train)

# AIC-based stepwise search
selected <- step(base,
                 scope = list(lower = y ~ 1,
                              upper = y ~ x1 + x2 + x3 + x4),
                 direction = "both", trace = FALSE)
summary(selected)

# Validate selected model on held-out data
pred <- predict(selected, newdata = test)
rmse <- sqrt(mean((test$y - pred)^2))
print(rmse)
```

Python: compact forward-selection template

```
import numpy as np
import statsmodels.api as sm

response = "y"
candidates = ["x1", "x2", "x3", "x4"]
selection_df = df  # Use training data here if doing train/test validation.
selected = []
```

```
current_fit = sm.OLS(selection_df[response],
                     np.ones((len(selection_df), 1))).fit()
current_aic = current_fit.aic
best_fit = current_fit

while candidates:
    trial_rows = []
    for var in candidates:
        cols = selected + [var]
        X_trial = sm.add_constant(selection_df[cols])
        fit = sm.OLS(selection_df[response], X_trial).fit()
        trial_rows.append((fit.aic, var, fit))

    best_aic, best_var, best_fit = min(trial_rows, key=lambda z: z[0])
    if best_aic < current_aic:
        selected.append(best_var)
        candidates.remove(best_var)
        current_aic = best_aic
    else:
        break

print("selected predictors:", selected)
print(best_fit.summary())
```

After any stepwise or subset search, validate the chosen model on data not used to select it when prediction matters.

Selection log template

```
Model-selection log
Question:
Candidate predictors considered:
Predictors excluded before fitting and why:
Selection method used:
Criterion used:
Validation method:
Final model formula:
Diagnostics checked after selection:
Limitations / post-selection cautions:
```

Defensibility check. Do not report selected-model p-values as if no search occurred. State how the model was selected and what independent validation, if any, was used after selection.

16 Regularization, PCA, and PCR

Purpose. Support Chapter 16 with compact workflows for ridge, LASSO, elastic net, PCA, and PCR. The main rule is train-only preprocessing: any scaling or principal-component fit learned from data must be learned on the training set and then applied to validation/test data.

Python: ridge, LASSO, and elastic net with train-only scaling

```
import numpy as np
import pandas as pd
from sklearn.linear_model import RidgeCV, LassoCV, ElasticNetCV, LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = df[["x1", "x2", "x3", "x4"]]
y = df["y"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1701
)

alphas = np.logspace(-3, 3, 80)
models = {
    "linear": make_pipeline(StandardScaler(), LinearRegression()),
    "ridge": make_pipeline(StandardScaler(), RidgeCV(alphas=alphas)),
    "lasso": make_pipeline(StandardScaler(), LassoCV(alphas=alphas, cv=5,
                                                     max_iter=10000)),
    "elastic_net": make_pipeline(StandardScaler(), ElasticNetCV(
        alphas=alphas, l1_ratio=[0.2, 0.5, 0.8, 1.0], cv=5, max_iter=10000
    ))
}

rows = []
for name, model in models.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({
        "model": name,
        "test_RMSE": np.sqrt(mean_squared_error(y_test, pred))
    })

print(pd.DataFrame(rows).sort_values("test_RMSE"))
```

Python: coefficient and selection summaries

```
# Coefficients from a fitted pipeline
```

```
lasso_pipe = models["lasso"]
coef = lasso_pipe.named_steps["lassocv"].coef_
coef_table = pd.DataFrame({"predictor": X.columns, "coef": coef})
print(coef_table.sort_values("coef"))
print("selected:", coef_table.loc[coef_table["coef"] != 0, "predictor"].tolist())
```

LASSO zeros are selection output, not scientific proof that a predictor has no effect. If inference is needed after LASSO selection, document the selection step and use post-selection caution.

R: ridge, LASSO, and elastic net with cross-validation

```
# install.packages("glmnet") if needed
library(glmnet)

X <- model.matrix(y ~ x1 + x2 + x3 + x4, data = df)[, -1]
y <- df$y

set.seed(1701)
idx <- sample(seq_len(nrow(X)), size = floor(0.75 * nrow(X)))
X_train <- X[idx, ]
X_test <- X[-idx, ]
y_train <- y[idx]
y_test <- y[-idx]

# glmnet standardizes predictors by default.
ridge_cv <- cv.glmnet(X_train, y_train, alpha = 0)
lasso_cv <- cv.glmnet(X_train, y_train, alpha = 1)
enet_cv <- cv.glmnet(X_train, y_train, alpha = 0.5)

pred_ridge <- predict(ridge_cv, newx = X_test, s = "lambda.min")
pred_lasso <- predict(lasso_cv, newx = X_test, s = "lambda.min")
pred_enet <- predict(enet_cv, newx = X_test, s = "lambda.min")

rmse <- function(y, pred) sqrt(mean((y - pred)^2))
c(
  ridge = rmse(y_test, pred_ridge),
  lasso = rmse(y_test, pred_lasso),
  elastic_net = rmse(y_test, pred_enet)
)
```

Python: PCA and PCR

```
import numpy as np
from sklearn.decomposition import PCA
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

# Assumes X_train, X_test, y_train, and y_test were created by a
# train/test split as in the regularization recipe above.
pcr = make_pipeline(
    StandardScaler(),
    PCA(),
    LinearRegression()
)

param_grid = {"pca__n_components": list(range(1, X_train.shape[1] + 1))}
grid = GridSearchCV(
    pcr, param_grid=param_grid,
    scoring="neg_root_mean_squared_error", cv=5
)
grid.fit(X_train, y_train)

pred_pcr = grid.predict(X_test)
print("best number of PCs:", grid.best_params_["pca__n_components"])
print("PCR test RMSE:", np.sqrt(mean_squared_error(y_test, pred_pcr)))
```

R: PCR pattern

```
# install.packages("pls") if needed
library(pls)

set.seed(1701)
idx <- sample(seq_len(nrow(df)), size = floor(0.75 * nrow(df)))
train_df <- df[idx, ]
test_df <- df[-idx, ]

pcr_fit <- pcr(y ~ x1 + x2 + x3 + x4, data = train_df,
               scale = TRUE, validation = "CV")
summary(pcr_fit)
validationplot(pcr_fit, val.type = "RMSEP")

# Choose ncomp from validation output, then predict on test data.
pred <- as.vector(predict(pcr_fit, newdata = test_df, ncomp = 2))
sqrt(mean((test_df$y - pred)^2))
```

Defensibility check. Report the tuning method, the selected penalty or number of components, validation performance, and what interpretability was lost or gained. Regularization and PCA/PCR can improve prediction stability, but they change coefficient interpretation.

17 Neural Network Regression Sanity Check

Purpose. Support Chapter 17 with a small prediction-benchmark workflow. Neural networks should be compared against simpler baselines, not treated as automatic replacements for interpretable models.

Python: shallow neural-network benchmark

```
import numpy as np
import pandas as pd
from sklearn.linear_model import RidgeCV
from sklearn.metrics import mean_squared_error
from sklearn.model_selection import train_test_split
from sklearn.neural_network import MLPRegressor
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X = df[["x1", "x2", "x3", "x4"]]
y = df["y"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1701
)

baseline = make_pipeline(
    StandardScaler(),
    RidgeCV(alphas=np.logspace(-3, 3, 80))
)
nnet = make_pipeline(
    StandardScaler(),
    MLPRegressor(
        hidden_layer_sizes=(20,),
        activation="relu",
        alpha=0.001,
        early_stopping=True,
        max_iter=2000,
        random_state=1701
    )
)

rows = []
for name, model in {"ridge_baseline": baseline, "shallow_nnet": nnet}.items():
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    rows.append({
        "model": name,
        "test_RMSE": np.sqrt(mean_squared_error(y_test, pred))
    })
```

```
print(pd.DataFrame(rows).sort_values("test_RMSE"))
```

R: shallow neural-network benchmark

```
# install.packages("nnet") if needed
library(nnet)

set.seed(1701)
idx <- sample(seq_len(nrow(df)), size = floor(0.75 * nrow(df)))
train <- df[idx, ]
test <- df[-idx, ]

predictors <- c("x1", "x2", "x3", "x4")
center <- sapply(train[predictors], mean)
scale_vals <- sapply(train[predictors], sd)
scale_vals[scale_vals == 0] <- 1

train_scaled <- train
test_scaled <- test
train_scaled[predictors] <- sweep(
  sweep(train[predictors], 2, center, "-"), 2, scale_vals, "/"
)
test_scaled[predictors] <- sweep(
  sweep(test[predictors], 2, center, "-"), 2, scale_vals, "/"
)

baseline <- lm(y ~ x1 + x2 + x3 + x4, data = train_scaled)
nnet_fit <- nnet(
  y ~ x1 + x2 + x3 + x4,
  data = train_scaled,
  size = 5,
  decay = 0.001,
  linout = TRUE,
  maxit = 1000,
  trace = FALSE
)

rmse <- function(actual, pred) sqrt(mean((actual - pred)^2))
baseline_pred <- predict(baseline, newdata = test_scaled)
nnet_pred <- predict(nnet_fit, newdata = test_scaled)

c(
  linear_baseline = rmse(test_scaled$y, baseline_pred),
  shallow_nnet = rmse(test_scaled$y, nnet_pred)
)
```

Over-tuning warning. Do not repeatedly change network size, activation, learning settings, or random seed until the test RMSE looks good. Use a validation set or cross-validation for tuning, then reserve a final test set for the final comparison when possible.

Briefing template. "The neural network improved validation RMSE by `...` compared with the interpretable baseline, at the cost of `....` Because the improvement is `small/moderate/large`, we recommend `....`"

Defensibility check. If the neural network is only slightly better than the baseline, the interpretable model may be preferable. If it is much better, investigate what nonlinear structure the simpler model missed.

18 End-to-End Mini Workflow

Purpose. Use this compact workflow when starting a lab or homework analysis. It is not meant to replace the detailed recipes above; it shows the order of operations.

Python pattern

```
import numpy as np
import pandas as pd
import statsmodels.api as sm
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error

# 1. Load
df = pd.read_csv("data/raw/my_data.csv")

# 2. Inspect and clean
df.columns = (
    df.columns.str.strip()
              .str.lower()
              .str.replace(" ", "_", regex=False)
)
print(df.info())
print(df.isna().mean().sort_values(ascending=False))

# Example cleaning decision
df.loc[df["weight_lb"] == 0, "weight_lb"] = np.nan

# Remove forbidden or leaky predictors before EDA/model selection
forbidden = ["id", "future_y", "post_outcome"]
df = df.drop(columns=[c for c in forbidden if c in df.columns])
```

```
model_df = df.dropna(subset=["y", "x1", "x2"])

# 3. EDA
model_df[["y", "x1", "x2"]].describe()
model_df[["y", "x1", "x2"]].corr()

# 4. Split for validation
train, test = train_test_split(model_df, test_size=0.25, random_state=1)

# 5. Fit baseline MLR
X_train = sm.add_constant(train[["x1", "x2"]])
y_train = train["y"]
fit = sm.OLS(y_train, X_train).fit()
print(fit.summary())

# 6. Diagnose training residuals
train = train.copy()
train["fitted"] = fit.fittedvalues
train["resid"] = fit.resid
plt.scatter(train["fitted"], train["resid"])
plt.axhline(0, linestyle="--")
plt.xlabel("Fitted values")
plt.ylabel("Residuals")
plt.show()

# 7. Validate on held-out data
X_test = sm.add_constant(test[["x1", "x2"]])
pred = fit.predict(X_test)
rmse = np.sqrt(mean_squared_error(test["y"], pred))
print("Validation RMSE:", rmse)
```

R pattern

```
# 1. Load
df <- read.csv("data/raw/my_data.csv")

# 2. Inspect and clean
names(df) <- tolower(trimws(names(df)))
names(df) <- gsub(" ", "_", names(df))
str(df)
colMeans(is.na(df))

df$weight_lb[df$weight_lb == 0] <- NA

# Remove forbidden or leaky predictors before EDA/model selection
forbidden <- c("id", "future_y", "post_outcome")
df <- df[, !(names(df) %in% forbidden)]
```

```
model_df <- df[complete.cases(df[, c("y", "x1", "x2")]), ]

# 3. EDA
summary(model_df[, c("y", "x1", "x2")])
cor(model_df[, c("y", "x1", "x2")], use = "complete.obs")

# 4. Split for validation
set.seed(1)
idx <- sample(seq_len(nrow(model_df)), size = floor(0.75 * nrow(model_df)))
train <- model_df[idx, ]
test <- model_df[-idx, ]

# 5. Fit baseline MLR
fit <- lm(y ~ x1 + x2, data = train)
summary(fit)

# 6. Diagnose training residuals
plot(fitted(fit), resid(fit))
abline(h = 0, lty = 2)

# 7. Validate on held-out data
pred <- predict(fit, newdata = test)
rmse <- sqrt(mean((test$y - pred)^2))
print(rmse)
```

Briefing sentence template. "We cleaned the data by ..., fit ... as a baseline model, checked residuals for ..., compared candidate models using ..., and concluded ... with the following limitations: ...."

19 Quick Troubleshooting Guide

Symptom	Likely issue	First check
Model will not run because of missing values	Required rows contain `NA` / `NaN`	Count missingness and decide drop/impute/flag
Coefficient is missing or marked not estimable	Design matrix is rank deficient	Check duplicate columns, dummy-variable trap, or perfectly related predictors
Large VIFs or unstable standard errors	Multicollinearity or redundant design columns	Check correlations, VIF, rank, condition number, and encoded design
Residual plot has a curve	Mean structure may be wrong	Try transformed predictors, polynomial terms, or interactions
Residual spread grows with fitted values	Nonconstant variance	Consider response transformation or a model with different variance structure
One point dominates the model	High leverage and/or influence	Inspect leverage, studentized residuals, and Cook's distance
Train error is low but test error is high	Overfitting or data leakage	Use simpler model, proper validation, and train-only preprocessing
Model chosen by search has impressive p-values	Post-selection inference problem	Report the search process and validate selected models
Regularized or neural-net model beats baseline slightly	Complexity may not be worth it	Compare validation RMSE gain with interpretability and compute cost
Correlation heatmap contains missing diagonal entries	Constant variable or insufficient variation	Check variance and remove non-informative constant columns

20 Cookbook Habits

A defensible coding workflow is not only about getting a model to run. In this course, every workflow should leave behind enough evidence that another analyst can reproduce and explain the result.

- Save the data-cleaning decisions, not just the cleaned data.
- Name derived columns so their meaning is clear.
- Keep model formulas visible.
- Extract residuals, fitted values, and leverage values when diagnostics matter.
- Compare candidate models on the same response scale.
- Keep train/test separation when estimating validation error.
- Report both the software output and the interpretation that follows from it.